

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL2- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	GIFTSON KEVIN MATTHEW.Y	Reg.No.:	192511167
Department:	B. Tech (CSE)	Date:	1/8/2026

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I / GIFTSON KEVIN MATTHEW. Y (192511167) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: GIFTSON KEVIN MATTHEW.Y

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

1. Debugging Signed Integer Addition Using 2's Complement

Introduction

In computer systems, signed integers are commonly represented using the **2's complement representation** because it simplifies arithmetic operations and allows the same hardware to perform both addition and subtraction. In an **8-bit 2's complement system**, the range of numbers that can be represented is **-128 to +127**. If the result of an arithmetic operation exceeds this range, the processor cannot represent the correct value, resulting in **overflow**. Therefore, detecting overflow is an important part of designing reliable arithmetic logic units (ALUs) in processors.

In the given problem, the program produces an incorrect result while adding **+120** and **+50** using 8-bit signed integers. To debug the issue, we need to analyze the binary addition, determine whether overflow has occurred, and identify an appropriate solution.

Step 1: Represent the Numbers in 8-bit 2's Complement

Since both numbers are positive, their binary representation is the same as ordinary binary.

Decimal Number	8-bit Binary Representation
+120	01111000
+50	00110010

Now perform the binary addition.

$$\begin{array}{r} 01111000 \text{ (+120)} \\ + 00110010 \text{ (+50)} \\ \hline 10101010 \end{array}$$

The obtained binary result is: 10101010

Step 2: Analyze the Result

The most significant bit (MSB) is **1**, which means the processor interprets the number as **negative** in 2's complement representation.

Finding its decimal value: $10101010_2 = -86_{10}$

However, mathematically,

$$120 + 50 = 170$$

The processor gives **-86** instead of **170**, indicating that something has gone wrong.

Step 3: Identify Whether Overflow Has Occurred

The range of an 8-bit signed integer is:

Minimum = -128

Maximum = +127

Since the actual result (**170**) is greater than **+127**, it cannot be represented in 8 bits.

This condition is called **signed overflow**.

Another simple way to detect overflow is by checking the signs of the operands and the result.

Overflow Rule

Overflow occurs when:

- Two positive numbers produce a negative result.
- Two negative numbers produce a positive result.

In this problem,

Operand 1: Positive (+120)

Operand 2: Positive (+50)

Result : Negative (-86)

Since **two positive numbers have produced a negative result**, overflow has definitely occurred.

Another hardware-based method is comparing the carry bits.

Carry into MSB $\neq$ Carry out of MSB

If these two carry values are different, the processor sets the **Overflow Flag (OF)**.

Therefore, both software analysis and hardware analysis confirm that overflow has occurred.

Step 4: Debugging the Issue

The binary addition itself is **correct**.

The error is **not in the addition process** but in the **limited size of the register**.

The processor stores only **8 bits**.

The actual binary representation of **170** requires more than the available signed range.

As a result, the processor stores only the lower 8 bits.

Actual Result = 170

Binary = 10101010

The processor interprets this stored value according to 2's complement rules, giving

10101010 = -86

Therefore, the incorrect answer is produced because the arithmetic result exceeds the storage capacity of the 8-bit signed register.

Step 5: Proposed Solution

To correctly detect and handle overflow, the processor or program should implement proper overflow detection mechanisms.

1. Check the Overflow Flag

Modern processors automatically generate an **Overflow Flag (OF)** whenever signed overflow occurs.

The program should examine this flag after every signed arithmetic operation.

If the flag is set, the program can display an error message or perform an alternative calculation.

2. Compare Operand and Result Signs

Overflow can also be detected using sign comparison.

Positive + Positive $\rightarrow$ Negative

Negative + Negative $\rightarrow$ Positive

If either condition occurs, overflow has taken place.

This method is widely used in software implementations.

3. Use a Larger Data Type

Instead of performing the calculation using an 8-bit integer, use a larger data type such as:

- 16-bit integer
- 32-bit integer
- 64-bit integer

For example,

$$120 + 50 = 170$$

A 16-bit signed integer has a range of
-32768 to +32767

Therefore, the value **170** can be represented without overflow.

4. Generate an Overflow Exception

In safety-critical applications such as banking systems, medical devices, and aircraft control systems, the processor should not ignore overflow.

Instead, it should:

- Stop the computation,
- Raise an overflow exception,
- Inform the programmer or operating system,
- Prevent incorrect results from affecting further calculations.

Advantages of Proper Overflow Detection

- Ensures arithmetic accuracy.
- Prevents incorrect calculations.
- Improves system reliability.
- Helps programmers identify errors quickly.
- Essential for embedded systems and real-time applications where incorrect numerical results can lead to serious failures.

Conclusion

The incorrect result obtained while adding **+120** and **+50** is caused by **signed overflow**, not by an error in the binary addition itself. The actual sum (**170**) exceeds the maximum value (**+127**) that can be represented in an 8-bit 2's complement system. Consequently, the processor stores an incorrect negative value (**-86**). This issue can be effectively handled by detecting overflow using the **Overflow Flag** or sign comparison and by performing calculations with a larger data type whenever necessary. Proper overflow detection and handling improve the accuracy, reliability, and robustness of computer systems, making them suitable for both general-purpose and critical real-world applications.

2. Debugging Performance Bottleneck in a Ripple Carry Adder and Optimizing Using a Carry Look-Ahead Adder

Introduction

Addition is one of the most frequently performed operations in a processor. The speed of arithmetic operations directly affects the overall performance of a computer system, especially in **real-time applications** such as embedded systems, robotics, medical devices, and industrial automation, where

quick response is essential. One of the simplest circuits used for binary addition is the **Ripple Carry Adder (RCA)**. Although it is easy to design and requires less hardware, its major disadvantage is the **carry propagation delay**. As the number of bits increases, the time required to complete an addition also increases, reducing the processor's performance.

In the given scenario, a processor using a Ripple Carry Adder experiences significant delays while executing arithmetic operations. To debug this issue, we must analyze how the carry propagates through each bit and then propose an optimized solution using a **Carry Look-Ahead Adder (CLA)**.

Understanding the Ripple Carry Adder

A Ripple Carry Adder is constructed by connecting multiple **Full Adders** in series. Each Full Adder adds two input bits along with the carry received from the previous stage.

For an **8-bit Ripple Carry Adder**, the carry moves as follows:

Carry

↓

Bit0 → Bit1 → Bit2 → Bit3 → Bit4 → Bit5 → Bit6 → Bit7

Each stage must wait for the carry generated by the previous stage before it can calculate its own sum and carry.

This sequential carry propagation is called the **Ripple Effect**.

Debugging the Performance Bottleneck

The major performance bottleneck lies in the way the carry signal propagates.

Suppose we add:

A = 11111111

B = 00000001

The carry generated at Bit 0 must travel through every Full Adder until it reaches the Most Significant Bit (MSB).

The sequence is:

Carry from Bit0

↓

Carry to Bit1

↓

Carry to Bit2

↓

Carry to Bit3

↓

Carry to Bit4

↓

Carry to Bit5

↓

Carry to Bit6

↓

Carry to Bit7

Since each Full Adder must wait for the previous carry, the total delay becomes the sum of all individual delays.

If one Full Adder takes **t nanoseconds**, then for an **n-bit Ripple Carry Adder**,

Total Delay = $n \times t$

For example,

If

Delay of one Full Adder = 2 ns

Then,

8-bit RCA Delay = $8 \times 2 = 16$ ns

Similarly,

32-bit RCA Delay = $32 \times 2 = 64$ ns

This clearly shows that as the number of bits increases, the delay also increases proportionally.

Reason for the Delay

The delay occurs because:

- Every Full Adder depends on the carry generated by the previous Full Adder.
- No stage can begin its computation until the previous carry is available.
- Carry propagation is completely sequential.

Therefore, the processor spends more time waiting for carries than performing the actual addition.

This delay becomes a serious issue in:

- Real-time systems
- High-speed processors
- Scientific computing
- Digital Signal Processing (DSP)
- Embedded controllers

Optimized Solution – Carry Look-Ahead Adder (CLA)

To eliminate the carry propagation delay, processors use a **Carry Look-Ahead Adder (CLA)**.

Unlike the Ripple Carry Adder, the CLA does **not wait** for the carry to ripple through each stage.

Instead, it predicts all carry values simultaneously using two signals:

1. Generate Signal (G)

A carry is generated when both input bits are 1.

$$G = A \times B$$

2. Propagate Signal (P)

A carry is propagated when either one of the input bits is 1.

$$P = A \oplus B$$

Using these signals, the carry is calculated as

$$C_{i+1} = G_i + P_i C_i$$

Since all Generate and Propagate signals are calculated at the same time, every carry bit is produced **in parallel**, instead of waiting for the previous one.

Comparison Between RCA and CLA

Feature	Ripple Carry Adder	Carry Look-Ahead Adder
Carry Generation	Sequential	Parallel
Speed	Slow	Very Fast
Carry Delay	High	Very Low
Hardware Complexity	Simple	More Complex
Cost	Low	Slightly Higher
Suitable For	Small circuits	High-speed processors

Advantages of Carry Look-Ahead Adder

The Carry Look-Ahead Adder offers several important advantages:

- Significantly reduces carry propagation delay.
- Performs arithmetic operations much faster.
- Improves processor execution speed.
- Suitable for high-performance CPUs and GPUs.
- Increases the efficiency of real-time systems.
- Reduces waiting time during arithmetic operations.
- Enhances overall system throughput.

Because of these advantages, modern processors prefer CLA or even more advanced adders instead of Ripple Carry Adders for high-speed computations.

Applications

Carry Look-Ahead Adders are widely used in:

- High-performance microprocessors
- Digital Signal Processors (DSP)
- Graphics Processing Units (GPUs)
- Supercomputers
- Embedded real-time systems
- Scientific computing applications

Conclusion

The performance bottleneck in the given processor is caused by the **sequential carry propagation** of the Ripple Carry Adder. Since every Full Adder must wait for the carry generated by the previous stage, the total delay increases with the number of bits, making the circuit unsuitable for real-time applications. This issue can be effectively solved by replacing the Ripple Carry Adder with a **Carry Look-Ahead Adder**, which calculates carry signals in parallel using **Generate** and **Propagate** functions. As a result, the delay is significantly reduced, arithmetic operations become much faster, and the overall performance of the processor is greatly improved. Therefore, the Carry Look-Ahead Adder is the preferred choice for modern high-speed computer architectures because it provides faster execution, better efficiency, and improved system reliability.

3. Debugging Incorrect Results in Booth's Multiplication Algorithm for Negative Operands

Introduction

Multiplication is one of the most important arithmetic operations performed by a processor. In computer architecture, **Booth's Algorithm** is widely used for **signed binary multiplication** because it efficiently handles both positive and negative numbers represented in **2's complement form**. Compared to ordinary binary multiplication, Booth's Algorithm reduces the number of addition and subtraction operations, making multiplication faster and more efficient.

However, if the algorithm is not implemented correctly, especially while handling **negative operands**, it may produce incorrect results. Common implementation errors include incorrect **bit-pair checking**, improper **arithmetic right shifting**, and faulty **sign extension**. In the given problem, the multiplication unit gives incorrect results for certain negative numbers. To debug this issue, we must carefully analyze each step of Booth's Algorithm and identify the possible source of error.

Understanding Booth's Algorithm

Booth's Algorithm multiplies two signed binary numbers by examining two bits at a time:

- Q_0 – Least Significant Bit (LSB) of the multiplier.
- Q_{-1} – An extra bit initially set to 0.

Depending on the values of Q_0 and Q_{-1} , one of the following operations is performed.

Q_0 Q_{-1}	Operation
00	No operation
01	Add Multiplicand (M)
10	Subtract Multiplicand (M)
11	No operation

After performing the required operation, the registers undergo an **Arithmetic Right Shift (ARS)**. This process is repeated for **n iterations**, where **n** is the number of bits.

Step-by-Step Debugging of the Algorithm

To identify why incorrect results occur, each stage of Booth's Algorithm should be verified carefully.

Step 1: Check the Bit-Pair Decision

The first step is to examine the pair Q_0 and Q_{-1} .

If the implementation interprets the bit pair incorrectly, the wrong arithmetic operation will be performed.

For example,

$$Q_0 \ Q_{-1} = 01$$

The algorithm should perform:

Accumulator = Accumulator + Multiplicand

If the hardware mistakenly performs subtraction instead of addition, every subsequent calculation becomes incorrect.

Similarly,

$Q_0 Q_{-1} = 10$

should always perform subtraction.

Therefore, incorrect bit-pair decoding is one of the most common causes of multiplication errors.

Step 2: Verify Arithmetic Right Shift

After every addition or subtraction, Booth's Algorithm performs an **Arithmetic Right Shift**.

Unlike a logical shift, an arithmetic shift **preserves the sign bit**.

Example:

Before Shift

11010100

After Arithmetic Right Shift

11101010

Notice that the leftmost **1** (sign bit) is copied.

If a **Logical Right Shift** is used instead,

11010100

becomes

01101010

The negative number is incorrectly converted into a positive number.

This completely changes the multiplication result.

Therefore, using the wrong type of shift is a major implementation error.

Step 3: Verify Sign Extension

Since Booth's Algorithm works with signed numbers represented in **2's complement**, the sign must always be preserved.

During shifting,

- Positive numbers extend with **0**
- Negative numbers extend with **1**

If zeros are inserted while shifting a negative number,

the sign changes, leading to incorrect multiplication results.

Proper sign extension ensures that negative operands retain their original value throughout the multiplication process.

Step 4: Verify the Number of Iterations

The algorithm must execute exactly **n iterations**, where **n** is the number of bits in the multiplier.

For example,

- 8-bit multiplier → 8 iterations
- 16-bit multiplier → 16 iterations

If the counter stops early or performs extra iterations, the final product will be incorrect.

Therefore, checking the loop counter is an important part of debugging Booth's Algorithm.

Possible Sources of Error

The multiplication unit may produce incorrect results due to one or more of the following reasons:

- Incorrect interpretation of the **Q_0-Q_{-1}** bit pair.

- Performing **addition instead of subtraction**, or vice versa.
- Using a **logical right shift** instead of an arithmetic right shift.
- Incorrect **sign extension** during shifting.
- Improper initialization of the **Q₋₁** bit.
- Executing fewer or more iterations than required.
- Incorrect handling of the accumulator during arithmetic operations.

Any one of these errors can produce an incorrect product, especially when multiplying negative numbers.

Suggested Corrections

To ensure accurate signed multiplication, the following corrections should be implemented:

1. Correct Bit-Pair Checking

The hardware or program should strictly follow Booth's decision table.

- **00** → No operation
- **01** → Add Multiplicand
- **10** → Subtract Multiplicand
- **11** → No operation

2. Always Perform Arithmetic Right Shift

Only **Arithmetic Right Shift (ARS)** should be used because it preserves the sign of the number. Logical shifting should never be used for signed multiplication.

3. Preserve Sign Extension

While shifting,

- Copy **1** for negative numbers.
- Copy **0** for positive numbers.

This ensures the value remains unchanged throughout the multiplication process.

4. Initialize Registers Correctly

Before starting the algorithm,

- Accumulator (A) should be initialized to **0**.
- **Q₋₁** should be initialized to **0**.
- Multiplicand and multiplier should be stored correctly in 2's complement form.

Incorrect initialization may lead to wrong intermediate calculations.

5. Execute the Correct Number of Cycles

The control unit should ensure that the multiplication process runs for exactly **n iterations**. This guarantees that every bit of the multiplier is processed correctly.

Advantages of Correct Implementation

A properly implemented Booth's Algorithm provides several benefits:

- Performs accurate multiplication for both positive and negative integers.
- Reduces the number of addition and subtraction operations.
- Improves processor performance.
- Efficiently handles consecutive 1's in the multiplier.
- Reduces hardware complexity compared to conventional multiplication methods.
- Widely used in modern processors and digital signal processors (DSPs).

Applications

Booth's Algorithm is commonly used in:

- Microprocessors
- Arithmetic Logic Units (ALUs)
- Digital Signal Processors (DSPs)
- Embedded systems
- Scientific computing
- Image and signal processing
- Computer graphics and multimedia applications

Conclusion

The incorrect results obtained during signed multiplication are mainly caused by implementation errors in **Booth's Algorithm**, particularly in **bit-pair checking, arithmetic right shifting, sign extension, and iteration control**. The algorithm itself is highly efficient for signed multiplication, but even a small mistake in these steps can produce incorrect outputs for negative operands. By correctly decoding the Q_0-Q_{-1} bit pairs, using **Arithmetic Right Shift**, preserving the **sign bit**, initializing the registers properly, and executing the required number of iterations, the multiplication unit can generate accurate and reliable results. Hence, careful debugging and correct implementation of Booth's Algorithm ensure efficient and error-free signed multiplication in modern computer architectures.

4. Debugging Inefficiency in the Restoring Division Algorithm and Optimizing Using the Non-Restoring Division Method

Introduction

Division is one of the fundamental arithmetic operations performed by a processor. In computer architecture, binary division is commonly implemented using algorithms such as **Restoring Division** and **Non-Restoring Division**. Among these, the **Restoring Division Algorithm** is simple to understand and implement, but it is not always efficient because it repeatedly performs restoration whenever the subtraction result becomes negative. This increases the number of arithmetic operations, leading to longer execution time and reduced processor performance.

In the given problem, an embedded system using the **Restoring Division Algorithm** experiences inefficiency due to repeated restoration steps. To debug this issue, it is necessary to analyze each stage of the algorithm, identify where unnecessary operations occur, and propose a more efficient solution using the **Non-Restoring Division Algorithm**.

Understanding the Restoring Division Algorithm

The Restoring Division Algorithm divides two binary numbers using repeated subtraction.

The main registers used are:

- **A (Accumulator)**: Stores the partial remainder.
- **Q (Quotient Register)**: Stores the dividend initially and the quotient finally.
- **M (Divisor Register)**: Stores the divisor.

For each iteration, the following steps are performed:

1. Shift the **A-Q** registers to the left.
2. Subtract the divisor (**M**) from the accumulator (**A**).
3. Check the sign of the result.

4. If the result is **positive or zero**, set the quotient bit to **1**.
5. If the result is **negative**, restore the previous value by adding the divisor back and set the quotient bit to **0**.
6. Repeat the process until all bits have been processed.

Although this algorithm always produces the correct quotient, the repeated restoration operation reduces its efficiency.

Debugging the Algorithm Workflow

To identify the cause of inefficiency, let us examine the workflow carefully.

Suppose after subtraction,

A = Negative

The algorithm immediately performs another operation:

$A = A + M$

This operation restores the original value of the accumulator.

Therefore, for every unsuccessful subtraction, two arithmetic operations are performed:

- One subtraction
- One restoration (addition)

As this process repeats many times during division, the processor spends extra clock cycles performing restoration instead of progressing toward the final result.

This repeated restoration is the primary performance bottleneck.

Identifying the Source of Inefficiency

The unnecessary operations occur whenever the accumulator becomes negative.

The sequence is:

Shift Left



Subtract Divisor



Negative Result?



Yes



Restore Previous Value



Continue Next Iteration

Each restoration requires an additional addition operation, increasing:

- Execution time
- CPU cycles
- Power consumption
- Hardware activity

In embedded systems where processors have limited computational resources and strict timing requirements, these extra operations significantly reduce overall performance.

Optimized Solution – Non-Restoring Division Algorithm

A more efficient approach is the **Non-Restoring Division Algorithm**.

Instead of restoring the accumulator immediately after a negative result, the algorithm **keeps the negative remainder** and corrects it during the next iteration.

The working principle is as follows:

- If the previous remainder is **positive**, subtract the divisor.
- If the previous remainder is **negative**, add the divisor.
- Continue the process without performing an immediate restoration.
- After all iterations are completed, perform a final correction only if necessary.

Thus, the repeated restoration step is eliminated.

How the Non-Restoring Method Improves Performance

The major improvement is that the processor avoids unnecessary addition operations after every negative subtraction.

Instead of this sequence:

Subtract



Negative



Restore

the algorithm performs:

Subtract



Negative



Continue Next Step

Only the final remainder is corrected if required.

As a result:

- Fewer arithmetic operations are performed.
- Processor execution becomes faster.
- Clock cycles are reduced.
- Overall efficiency improves.

This makes the Non-Restoring Division Algorithm more suitable for high-speed processors and embedded systems.

Comparison Between Restoring and Non-Restoring Division

Feature	Restoring Division	Non-Restoring Division
Restoration Step	Required after every negative result	Not required during intermediate steps
Number of Arithmetic Operations	Higher	Lower
Execution Speed	Slower	Faster
Hardware Efficiency	Lower	Higher
CPU Clock Cycles	More	Fewer
Suitable For	Simple processor designs	High-speed and embedded processors

Advantages of the Non-Restoring Division Algorithm

The Non-Restoring Division method offers several advantages:

- Eliminates repeated restoration operations.
- Reduces the total number of arithmetic calculations.
- Improves processor speed and efficiency.
- Requires fewer clock cycles to complete division.
- Reduces power consumption in embedded systems.
- Increases overall throughput of the Arithmetic Logic Unit (ALU).
- Provides better performance in real-time applications.

Because of these benefits, modern processors often prefer the Non-Restoring Division Algorithm over the traditional Restoring Division Algorithm.

Applications

The Non-Restoring Division Algorithm is widely used in:

- Embedded systems
- Microprocessors
- Digital Signal Processors (DSPs)
- Arithmetic Logic Units (ALUs)
- Scientific computing
- Signal processing applications
- High-speed computing systems

Conclusion

The inefficiency in the given embedded system is mainly caused by the **repeated restoration steps** performed by the **Restoring Division Algorithm** whenever a subtraction produces a negative remainder. These additional restoration operations increase the number of arithmetic calculations, consume more clock cycles, and reduce the overall performance of the processor. This issue can be effectively solved by replacing the Restoring Division Algorithm with the **Non-Restoring Division Algorithm**, which avoids immediate restoration and performs correction only when necessary. As a result, the number of arithmetic operations is reduced, execution time becomes shorter, processor efficiency improves, and the system achieves faster and more reliable division. Therefore, the Non-Restoring Division Algorithm is considered a more efficient and practical solution for modern embedded and real-time computer systems.

5. Debugging Inaccuracy in IEEE 754 Single Precision Floating-Point Addition and Optimizing for Better Accuracy

Introduction

Floating-point numbers are widely used in computer systems to represent **real numbers** that contain fractional values. Applications such as **scientific computing, artificial intelligence, machine learning, engineering simulations, financial analysis, and graphics processing** depend on floating-point arithmetic for accurate numerical computations. The **IEEE 754 standard** is the most widely accepted format for representing floating-point numbers because it provides a standardized method for storing and performing arithmetic operations.

In **single precision (32-bit)** IEEE 754 format, a floating-point number consists of a **1-bit sign**, an **8-bit exponent**, and a **23-bit mantissa (fraction)**. Although this representation is efficient and memory-friendly, it has limited precision. When a very small number is added to a very large number, the result

may become inaccurate due to **exponent alignment, normalization, and rounding errors**. Therefore, debugging these issues is essential to ensure reliable numerical computations.

Understanding IEEE 754 Single Precision Format

A 32-bit floating-point number is divided into three fields:

Field	Number of Bits	Purpose
Sign Bit	1	Represents positive or negative value
Exponent	8	Determines the scale or magnitude of the number
Mantissa (Fraction)	23	Stores the significant digits of the number

The value of a floating-point number is calculated as:

$$\text{Value} = (-1)^{\text{Sign}} \times 1.\text{Mantissa} \times 2^{(\text{Exponent}-127)}$$

This representation allows a very wide range of numbers to be stored, but the limited mantissa size can lead to precision loss during arithmetic operations.

Debugging the Floating-Point Addition Process

To understand the source of the error, let us examine what happens when a **very large number** is added to a **very small number**.

Example:

1,000,000.0 + 0.000001

Mathematically, the answer should be:

1000000.000001

However, in IEEE 754 single precision, the processor may produce:

1000000.000000

The smaller value appears to disappear.

This is not a programming mistake but a limitation of floating-point representation.

Step 1: Exponent Alignment

Before addition, both floating-point numbers must have the **same exponent**.

To achieve this, the processor shifts the mantissa of the smaller number to the right until both exponents match.

Example:

Large Number

1.111010×2^{20}

Small Number

1.000001×2^0

To align the exponents, the smaller mantissa is shifted many positions to the right.

After repeated shifting,

[illegible]

Most of its significant bits are lost.

As a result, the smaller number contributes very little—or sometimes nothing—to the final answer.

This is called **loss of precision**.

Step 2: Normalization

After the addition is completed, the result must be converted into the standard IEEE 754 format.

This process is called **normalization**.

During normalization:

- The mantissa is shifted left or right.
- The exponent is adjusted accordingly.

Although normalization ensures that the floating-point number follows the IEEE 754 standard, it may discard additional bits, leading to further precision loss.

Step 3: Rounding Errors

Since the mantissa has only **23 bits**, it cannot store every decimal value exactly.

If extra bits remain after normalization, the processor rounds the value according to the IEEE 754 rounding rules.

For example,

Stored Value

1.1010101010101010101010101010111...

Only the first **23 bits** are stored.

The remaining bits are rounded off.

Repeated arithmetic operations cause these rounding errors to accumulate, resulting in noticeable inaccuracies in scientific calculations.

Reason for Inaccurate Results

The main causes of inaccuracy are:

- **Exponent Alignment:** Small numbers lose significant bits while matching the exponent of larger numbers.
- **Limited Mantissa:** Only 23 bits are available to represent the fractional part.
- **Normalization:** Additional shifting may reduce precision.
- **Rounding:** Extra bits are discarded to fit the IEEE 754 format.
- **Finite Precision:** Not all real numbers can be represented exactly in binary.

Therefore, adding numbers with a large difference in magnitude often leads to loss of precision.

Optimization Techniques

To improve numerical accuracy, the following optimization techniques can be used.

1. Use Double Precision

Instead of single precision, use **IEEE 754 Double Precision (64-bit)**.

Feature	Single Precision	Double Precision
Total Bits	32	64
Exponent Bits	8	11
Mantissa Bits	23	52

A larger mantissa stores more significant digits, greatly reducing rounding errors and improving the accuracy of arithmetic operations.

2. Use Numerically Stable Algorithms

Algorithms designed for numerical stability can minimize floating-point errors.

Examples include:

- **Kahan Summation Algorithm**
- **Pairwise Summation**

These algorithms compensate for small rounding errors and provide more accurate results when adding many floating-point values.

3. Reorder Arithmetic Operations

Instead of adding very small numbers directly to a very large number, first add numbers of similar magnitude.

For example:

Instead of:

$1000000 + 0.000001 + 0.000002$

Use:

$0.000001 + 0.000002 = 0.000003$

Then,

$1000000 + 0.000003$

This reduces the loss of precision during exponent alignment.

4. Avoid Unnecessary Floating-Point Operations

Repeated floating-point calculations increase cumulative rounding errors.

Where possible:

- Reduce intermediate calculations.
- Store values with higher precision.
- Avoid repeated conversions between integer and floating-point formats.

These practices improve the overall accuracy of scientific applications.

Advantages of the Proposed Optimizations

Using the above optimization techniques provides several benefits:

- Improves numerical accuracy.
- Reduces rounding and precision errors.
- Produces more reliable scientific calculations.
- Enhances the performance of engineering and simulation software.
- Increases the stability of numerical algorithms.
- Ensures better results in AI, machine learning, graphics, and financial applications.

Applications

Accurate floating-point arithmetic is essential in:

- Scientific research
- Artificial Intelligence (AI)
- Machine Learning
- Engineering simulations

- Medical imaging systems
- Weather forecasting
- Computer graphics and gaming
- Financial and banking software
- Aerospace and space exploration

Conclusion

The inaccurate results produced during IEEE 754 single-precision floating-point addition are mainly caused by **exponent alignment, normalization, limited mantissa size, and rounding errors**. When a very small number is added to a very large number, the smaller value may lose its significant bits during exponent alignment, causing a loss of precision. This issue can be effectively minimized by using **double precision**, applying **numerically stable algorithms** such as **Kahan Summation**, reordering arithmetic operations, and reducing unnecessary floating-point calculations. These optimization techniques significantly improve numerical accuracy, making scientific and engineering applications more reliable. Therefore, while IEEE 754 single precision offers efficient memory usage, careful handling of floating-point operations is essential to achieve accurate and dependable computational results in modern computer systems.